Scientific Computation

An Out-of-Order History of Experimental Science

Bill Cochran
wkcochran@gmail.com
https://github.com/wkcochran123/measurement

July 9, 2026

Preface

This is the reading gauge. Volumes 1 through 3 build the instrument — in mathematics, in physics, in computation. This volume explains it, to anyone, and hands you the means to check it yourself. It reads as an out-of-order history of experimental science: the ideas are arranged by how the machine uses them, not by when they were discovered.

The method is a discipline of decorrelation. Each result is told four ways that share no vocabulary — a mathematical account that uses no mathematics, a physical account that uses no computation, a computational account that uses no physics, and a literal walk through the code. Three registers that cannot borrow each other's words, plus the code itself: their agreement is the argument. When four descriptions that could not have coordinated land on the same thing, the thing is hard to fake.

There is one rule of honesty here, and it is carried in the citations rather than announced. Where the text leans on settled science it names, generously, the people whose experiments and theorems it stands on, so the reader rests on known ground. Where the citations thin out, something new is happening, with no prior name to cite; that silence is the signal to slow down and check. Generosity, then, is an instrument — it makes the novel legible by contrast.

The chapter that follows is the pilot: the fine-structure result, walked through as the literal Lean, told plainly enough to re-run. The tone stays light throughout; the volume opens, elsewhere, on a doesn't-matter particle hunting for posit-wrong ones, and never entirely puts the joke down —

because the joke keeps turning out to be the structure.

Contents

Part I

The Reading

Chapter 1

The Bound

I have the four artifacts and verified the bound arithmetically ($10^{18}/2^{60}$ floors to 0, so `dedekindHalt` halts at $n = 60$). Here is the reading.

Open `device/Measurement/AlphaCapstone.lean`. Everything else is scaffolding for the one line that lives there:

```
theorem one_eq_point_nine_repeating_count_to_three :
    loopRem 2 4 = 0
    ∧ holonomy flatPath   (pairVariation node1 node2) = -1
    ∧ holonomy tiltedPath (pairVariation node2 node1) =  0
    ∧ holonomy tiltedPath (pairVariation node1 node2) =  1
```

Read it aloud as four claims. The residue closes to null — $1 = 0.999\ldots$, discharged `by decide`, the standard kernel-checked decision procedure you'd reach for in any Lean proof. Then three holonomy readings: -1, 0, $+1$. The electron, the null identity, the positron — count to three, and no fourth. The `holonomy` function and its `pairVariation` argument aren't imported from `Mathlib`; they're built here, in-house. That's the tell: where the citations run out, the new thing is happening. Pay attention to those lines.

Now the line that makes it a theorem and not a story. The last line of the file is `#print axioms one_eq_point_nine_repeating_count_to_three`,

and it prints `[]`. Empty. Not `[Classical.choice]`, not even the usual `[propext, Quot.sound]` most Lean developments carry — nothing. The entire **structure** of the coupling — the electron's state -1, the three states, the count-to-three, and the dimensionlessness that follows from the residue closing rather than resting on a borrowed unit — is derived on an empty axiom footprint. That is a fact off the build. Re-run the print yourself; it doesn't need your trust.

Next, `device/Measurement/VerifyAPI.lean` — the product surface, the part meant to be run. `verify (ekg) (claimed)` takes a claimed calibration, runs the deterministic experiment `EKG.outgrown?`, and returns a `VerifyResult`: `verified : Bool`, and a `residual` that lands either `.aboveFloor` (claimable) or `.belowFloorAntimatter`. That second constructor is the positron pun made literal — a claim that outgrows the budget is cut below the floor, into antimatter, unclaimable. The wrapper is deliberately thin over the already-built engine. The point isn't the code; the point is that it's a function you can call. `#eval verify EKG.raw flagshipClaim`, again, tomorrow, on your machine. Determinism is the whole ethic here: repeatability *is* the honesty. Nobody is asking you to believe the reading. They're handing you the instrument and asking you to check it.

Now `RichardsonSelfNaming.lean`, and this is the novel move, so slow down. `alphaAtLevel n :=` $10^{18}/2^n$ — the construction's own per-level self-naming value, scaled for exact integer arithmetic. `residueAtLevel n := alphaAtLevel n - alphaAtLevel (n+1)` — how much the name has *not yet* closed to the thing. And `selfNamingExtrapolant :=` $2 \cdot \alpha(n{+}1) - \alpha(n)$, the extrapolation of the name "electron" to the thing electron, its own fixed point. Interpret this — and I mark it as interpretation — as the electron's self-energy: ask the electron its number and it reports its state for free; the coupling is what it *charges* for the self-embrace, the electron paying to be itself. The order $p = 1$ isn't tuned to a target; it's forced by the observed halving $\rho = \frac{1}{2}$. Choosing the order to hit a number would be fishing in a

Richardson costume. It refuses to.

Finally `AlphaDedekind.lean`, the bound drawn to scale. `dedekindHalt` is ordinary structural recursion on fuel — standard Lean, nothing exotic — writing the nines of $0.999\ldots$ one bit per step and halting the first step the residue drops below `floorEps := 1`. Then:

```
def haltIndex : Nat := dedekindHalt 0 512
```

`#eval haltIndex` prints **60**. You can check the arithmetic by hand: $10^{18} \approx 2^{59.79}$, so `alphaAtLevel 59` $= 1$, `alphaAtLevel 60` $= 0$, and the residue first vanishes at $n = 60$. Sixty bits. That's the bound, and `#print axioms haltIndex` is again `[]`.

The nines it writes are the structure it can see. The place it stops is the floor. The tail it cannot write is the magnitude. Interpretively — Ricci trace versus Weyl shape, marked as reading, not proof — a machine that counts volume is blind to pure shape, and the size of the coupling lives in the shape. So 137.036 appears nowhere in the reading path. It is only what a Penning trap reads off in the lab: the value the machine *proved* it was structurally blind to. Not missing. Refused, and the refusal is the theorem.

The four `#eval`s and the four `#print axioms` are the whole argument. Run them. That's the honesty.